

Projektowanie i Analiza Algorytmów	
Kierunek <i>Informatyczne Systemy Automatyki</i>	Termin <i>wtorek 17⁰⁵ – 18⁴⁵</i>
Imię, nazwisko, numer albumu <i>Jan Hulbój 284197</i>	Data <i>14.04.2026</i>
Link do projektu https://github.com/mlody-jano/algorithm_design_and_analysis	



Politechnika
Wrocławska

SPRAWOZDANIE – MINIPROJEKT NR 2

Poniższy dokument jest kompleksowym sprawozdaniem opisującym badania przeprowadzone w ramach pierwszego projektu na przedmiocie **Projektowanie i Analiza Algorytmów**. Przedmiotem pierwszego projektu były **algorytmy grafowe**, gdzie zaimplementowano algorytm znajdowania najkrótszej drogi Bellmana-Forda, a następnie zbadano jego złożoność obliczeniową. Ponadto, zweryfikowano złożoność obliczeniową algorytmu dla dwóch różnych implementacji grafu tj. w postaci **listy sąsiedztwa** oraz **macierzy sąsiedztwa**. Dane uzyskane podczas prowadzenia testów, zostały zaprezentowane w postaci wykresów oraz tabel. Testowe dane zostały uzyskane poprzez klasę **GraphGenerator**, która jest odpowiedzialna za tworzenie testowych grafów.

Spis treści

1	Wstęp teoretyczny	2
1.1	Algorytm Bellmana-Forda	2
2	Założenia projektowe	2
3	Wyniki badań	3
3.1	Implementacja na liście sąsiedztwa	3
3.2	Implementacja na macierzy sąsiedztwa	4
3.3	Wyniki dla różnych gęstości	5
3.3.1	Gęstość dens = 25%	5
3.3.2	Gęstość dens = 50%	6
3.3.3	Gęstość dens = 75%	6
3.3.4	Gęstość dens = 100%	7
4	Wnioski	7

Spis rysunków

1	Fragment kodu odpowiedzialny za pomiar czasu algorytmów grafowych.	3
2	Dodatkowe deklaracje wykorzystywane przez pomiar czasu.	3
3	Złożoność obliczeniowa algorytmu dla implementacji na listy sąsiedztwa	4
4	Złożoność obliczeniowa algorytmu dla implementacji na macierzy sąsiedztwa	5
5	Złożoność obliczeniowa algorytmu dla gęstości grafu dens = 25%.	5
6	Złożoność obliczeniowa algorytmu dla gęstości grafu dens = 50%.	6
7	Złożoność obliczeniowa algorytmu dla gęstości grafu dens = 75%.	7
8	Złożoność obliczeniowa algorytmu dla gęstości grafu dens = 100%.	7

Spis tabel

1	Złożoności obliczeniowe algorytmu Bellmana-Forda()	2
2	Wyniki uzyskane podczas testów algorytmu Bellmana-Forda dla implementacji grafu w postaci listy sąsiedztwa.	3
3	Wyniki uzyskane podczas testów algorytmu Bellmana-Forda dla implementacji grafu w postaci macierzy sąsiedztwa.	4
4	Porównanie wyników działania algorytmu Bellmana-Forda dla dwóch różnych implementacji grafu przy gęstości równej 25%	5
5	Porównanie wyników działania algorytmu Bellmana-Forda dla dwóch różnych implementacji grafu przy gęstości 50%	6
6	Porównanie wyników działania algorytmu Bellmana-Forda dla dwóch różnych implementacji grafu przy gęstości 75%	6
7	Porównanie wyników działania algorytmu Bellmana-Forda dla dwóch różnych implementacji grafu przy gęstości 100%	7

1 Wstęp teoretyczny

1.1 Algorytm Bellmana-Forda

Algorytm Bellmana-Forda jest jednym z algorytmów grafowych pozwalających na odnalezienie najkrótszej drogi pomiędzy dwoma wierzchołkami. Opiera on się na metodzie **relaksacji**, tj. sprawdzaniu czy po przejściu daną krawędzią, nie otrzymamy krótszej niż dotychczas ścieżki. Algorytm iteruje po $|V| - 1$ wierzchołkach i sprawdza drogi do dostępnych wierzchołków. Złożoność obliczeniowa tej operacji to $O(|V|)$. Po wybraniu wierzchołka, algorytm ponownie dokonuje iteracji, lecz tym razem po krawędziach wychodzących z wierzchołka, co ponownie skutkuje złożonością obliczeniową $O(|E|)$. Gdy znajdzie ścieżkę do wybranego wierzchołka, która jest krótsza niż dotychczasowa, nadpisuje ją. W jednej iteracji algorytm sprawdzi **każdą** krawędź, pod kątem aktualizacji najkrótszej drogi.

Optymistyczna złożoność obliczeniowa algorytmu zakłada scenariusz w którym każdy z wierzchołków ma tylko jedną krawędź wychodzącą, co sprawia że przeszukiwanie listy krawędzi staje się operacją w czasie stałym, a więc wykonywana jest tylko iteracja po $|V| - 1$ wierzchołków. W rzeczywistości jest to przypadek mało prawdopodobny i niepraktyczny, ponieważ graf w którym każdy z wierzchołków ma tylko jedną krawędź, jest listą, dla której istnieje tylko jedna ścieżka przechodzenia przez wierzchołki.

Oczekiwane złożoności obliczeniowe algorytmu **Bellmana-Forda** wynoszą:

Tabela 1: Złożoności obliczeniowe algorytmu **Bellmana-Forda()**

Lp.	Nazwa	Optymistyczna	Średnia	Pesymistyczna
1.	bellmanFord()	$O(V)$	$O(V \cdot E)$	$O(V \cdot E)$

2 Założenia projektowe

W celu zapewnienia odpowiedniego podłoża merytorycznego do wyciągnięcia wniosków dotyczących badanych algorytmów, przyjęte zostały następujące założenia:

- **Rozmiary struktur: ilości wierzchołków:** 10 50 100 250 500; **gęstości grafu:** 25%, 50%, 75%, 100%
- **Ilość powtórzeń operacji:** 100
- **Sposób pomiaru czasu:** Pomiar czasu z wykorzystaniem `high_resolution_clock`.

Testy utworzonych struktur danych były prowadzone na urządzeniu wyposażonym w następujące komponenty:

- **Procesor** - AMD Ryzen 7 2700X 3,7Ghz
- **RAM** - 16GB

Badanie czasu operacji zostało zrealizowane w postaci wykorzystania biblioteki **chrono**, która zawiera implementację klasy **high_resolution_clock**, która swoją funkcją **now()**, pozwala na dokładny pomiar czasu pracy algorytmów.

```
auto t0 = Clock::now();
auto bf = bellmanFord(*graph, source);
double timeMs = Ms(Clock::now() - t0).count();
```

Rysunek 1: Fragment kodu odpowiedzialny za pomiar czasu algorytmów grafowych.

```
using Clock = std::chrono::high_resolution_clock;
using Ms = std::chrono::duration<double, std::milli>;
```

Rysunek 2: Dodatkowe deklaracje wykorzystywane przez pomiar czasu.

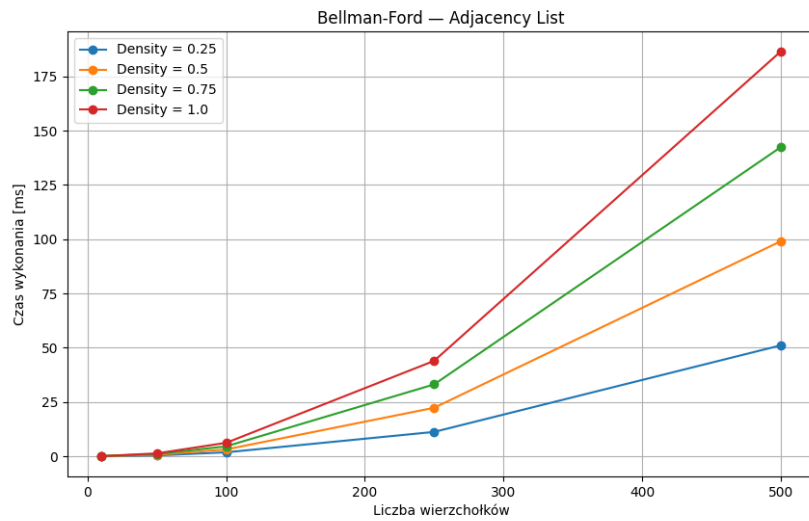
3 Wyniki badań

3.1 Implementacja na liście sąsiedztwa

Wyniki badań prowadzonych dla algorytmu **Bellmana-Forda** na grafie zaimplementowanym przez **listę sąsiedztwa** zostały zaprezentowane na poniższej tabeli oraz wykresie:

Tabela 2: Wyniki uzyskane podczas testów algorytmu Bellmana-Forda dla implementacji grafu w postaci listy sąsiedztwa.

V = 10			V = 50			V = 100		
Lp.	Density	Time [ms]	Lp.	Density	Time [ms]	Lp.	Density	Time [ms]
1.	25%	0,0278	1.	25%	0,3874	1.	25%	1,8113
2.	50%	0,0302	2.	50%	0,6816	2.	50%	3,1376
3.	75%	0,0506	3.	75%	1,0194	3.	75%	4,5679
4.	100%	0,0576	4.	100%	1,3212	4.	100%	6,1910
V = 250			V = 500					
Lp.	Density	Time [ms]	Lp.	Density	Time [ms]			
1.	25%	11,2131	1.	25%	51,0689			
2.	50%	22,3199	2.	50%	99,0230			
3.	75%	33,0962	3.	75%	142,3500			
4.	100%	43,9408	4.	100%	186,4211			



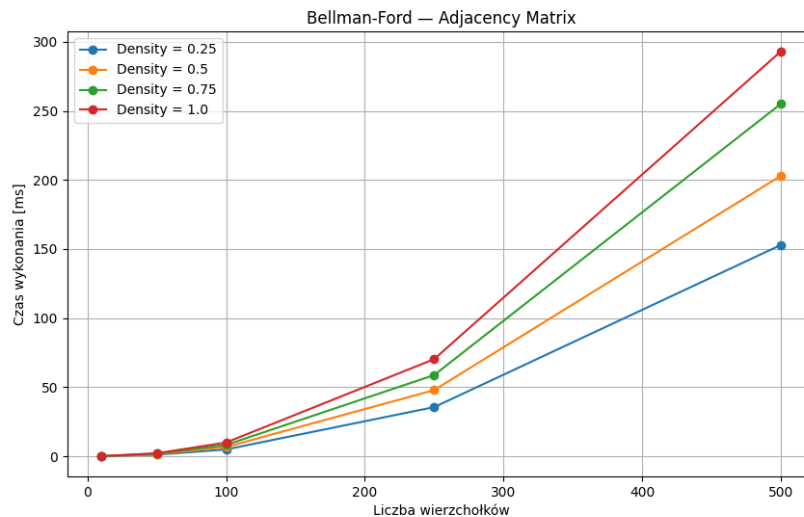
Rysunek 3: Złożoność obliczeniowa algorytmu dla implementacji na **listy sąsiedztwa**.

3.2 Implementacja na macierzy sąsiedztwa

Wyniki badań prowadzonych dla algorytmu **Bellmana-Forda** na grafie zaimplementowanym przez **macierz sąsiedztwa** zostały zaprezentowane na poniższej tabeli oraz wykresie:

Tabela 3: Wyniki uzyskane podczas testów algorytmu Bellmana-Forda dla implementacji grafu w postaci macierzy sąsiedztwa.

V = 10			V = 50			V = 100		
Lp.	Density	Time [ms]	Lp.	Density	Time [ms]	Lp.	Density	Time [ms]
1.	25%	0,0579	1.	25%	1,1499	1.	25%	4,9430
2.	50%	0,0693	2.	50%	1,4524	2.	50%	6,7169
3.	75%	0,0822	3.	75%	1,9147	3.	75%	8,2319
4.	100%	0,0923	4.	100%	2,1894	4.	100%	9,9941
V = 250			V = 500					
Lp.	Density	Time [ms]	Lp.	Density	Time [ms]			
1.	25%	35,5580	1.	25%	152,8711			
2.	50%	47,8731	2.	50%	202,9074			
3.	75%	58,7693	3.	75%	254,9973			
4.	100%	70,2722	4.	100%	292,9343			



Rysunek 4: Złożoność obliczeniowa algorytmu dla implementacji na **macierzy sąsiedztwa**.

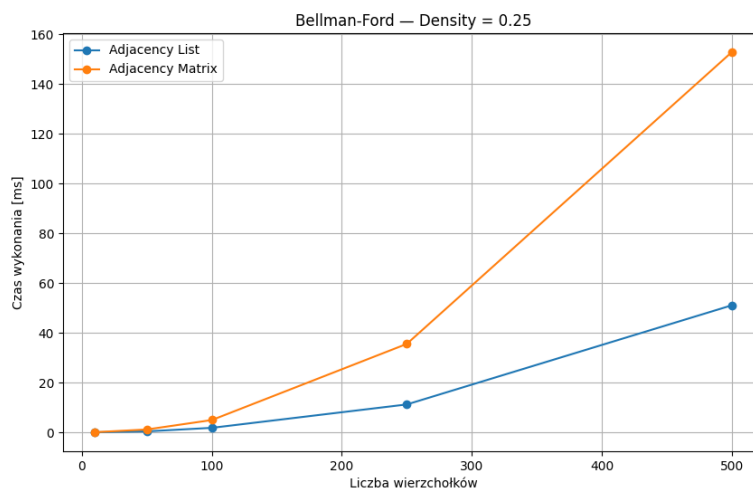
3.3 Wyniki dla różnych gęstości

Poniższe wykresy stanowią porównanie złożoności obliczeniowej działania algorytmu **Bellmana-Forda** dla każdej z zadanych gęstości grafu. Dane zostały przedstawione w postaci wykresów i tabel.

3.3.1 Gęstość dens = 25%

Tabela 4: Porównanie wyników działania algorytmu Bellmana-Forda dla dwóch różnych implementacji grafu przy gęstości równej 25%

Lp.	V	Adjacency List	Adjacency Matrix
1.	10	0,0278	0,0579
2.	50	0,3874	1,1499
3.	100	1,8133	4,9430
4.	250	11,2131	35,5580
5.	500	51,0689	152,8711

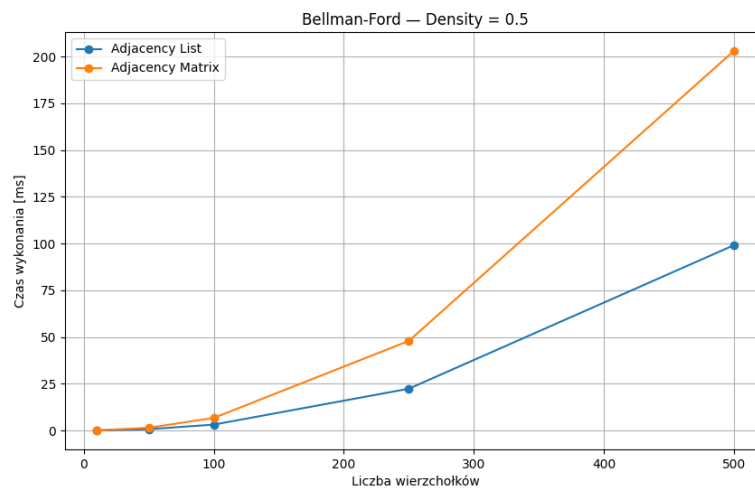


Rysunek 5: Złożoność obliczeniowa algorytmu dla gęstości grafu $dens = 25\%$.

3.3.2 Gęstość dens = 50%

Tabela 5: Porównanie wyników działania algorytmu Bellmana-Forda dla dwóch różnych implementacji grafu przy gęstości 50%

Lp.	V	Adjacency List	Adjacency Matrix
1.	10	0,0302	0,0693
2.	50	0,6816	1,4524
3.	100	3,1376	6,7169
4.	250	22,3199	47,8731
5.	500	99,0230	202,9074

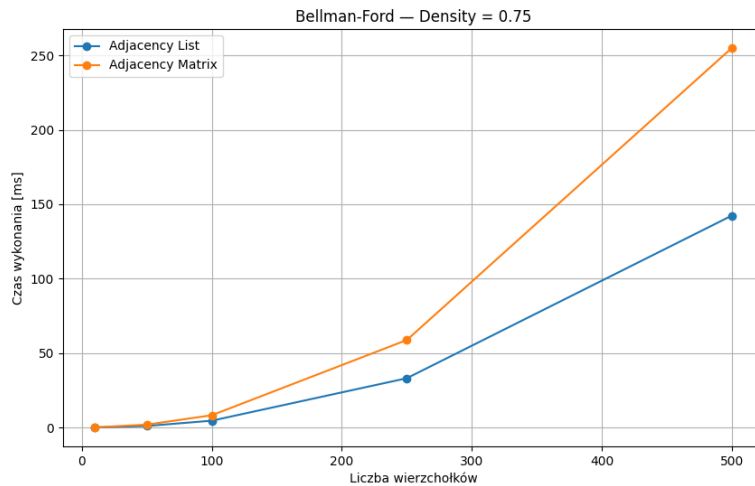


Rysunek 6: Złożoność obliczeniowa algorytmu dla gęstości grafu $dens = 50\%$.

3.3.3 Gęstość dens = 75%

Tabela 6: Porównanie wyników działania algorytmu Bellmana-Forda dla dwóch różnych implementacji grafu przy gęstości 75%

Lp.	V	Adjacency List	Adjacency Matrix
1.	10	0,0506	0,0822
2.	50	1,0194	1,9147
3.	100	4,5679	8,2319
4.	250	33,0962	58,7693
5.	500	142,3500	254,9973

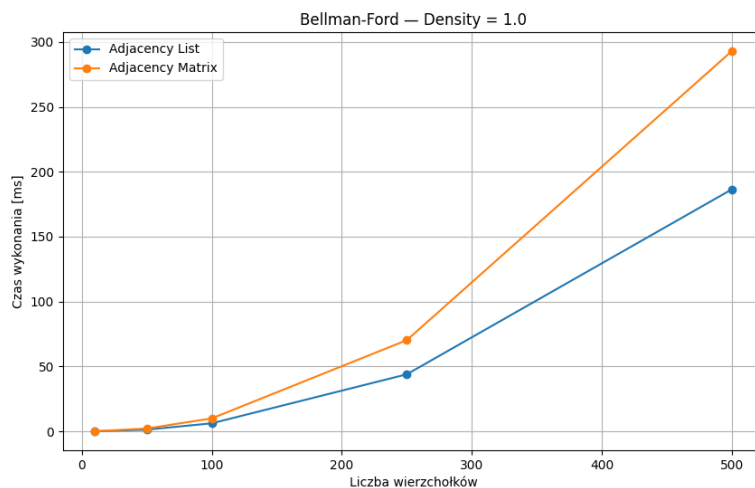


Rysunek 7: Złożoność obliczeniowa algorytmu dla gęstości grafu $dens = 75\%$.

3.3.4 Gęstość $dens = 100\%$

Tabela 7: Porównanie wyników działania algorytmu Bellmana-Forda dla dwóch różnych implementacji grafu przy gęstości 100%

Lp.	$ V $	Adjacency List	Adjacency Matrix
1.	10	0,0576	0,0923
2.	50	1,3212	2,1894
3.	100	6,1910	9,9941
4.	250	43,9408	70,2722
5.	500	186,4211	292,9343



Rysunek 8: Złożoność obliczeniowa algorytmu dla gęstości grafu $dens = 100\%$.

4 Wnioski

Przeprowadzone badania i osiągnięte wyniki stworzyły jasne i stanowcze podłoże merytoryczne do wyciągnięcia należytych wniosków na temat algorytmu Bellmana-Forda oraz jego działania na różnych implementacjach struktury grafowej.

Implementacja grafu na liście sąsiedztwa okazała się lepsza niż implementacja macierzowa w każdej konfiguracji. Jest to konsekwencją znacznie szybszego dostępu do krawędzi wierzchołków, co jest kluczową operacją w algorytmach grafowych. Lista sąsiedztwa pozwala na liniowy $O(1)$ dostęp do krawędzi wierzchołka, podczas gdy macierz sąsiedztwa wymaga kwadratowego czasu $O(n^2)$ na znalezienie krawędzi. Dopiero przy gęstych grafach o większej ilości wierzchołków macierz sąsiedztwa zbliża się narzutem czasowym do implementacji poprzez listę sąsiedztwa.

Na podstawie wykresów złożoności obliczeniowej algorytmu Bellmana-Forda w funkcji gęstości grafu można jasno zauważyć, że uzyskane wyniki są zależne od ilości krawędzi w badanych grafach, tj. od gęstości grafu. Potwierdza to zakładaną złożoność obliczeniową $O(|V| \cdot |E|)$.

Bibliografia

1. GOODRICH, Michael T.; TAMASSIA, Roberto; MOUNT, David M. *Data Structures and Algorithms in C++*. 2nd ed. Hoboken, N.J: Wiley, 2011. ISBN 978-0-470-38327-8.